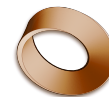


Name: _____

Class: _____

Mr. Rawson • GCSE Computer Science



Year 11 2026

Programming 1 — Answers

The challenge program and the “break it on purpose” table — OCR J277 §2.2 and §2.3.3

Read this first — your answer probably looks different, and that is fine

There is no single correct answer to this challenge. There are at least four genuinely different ways to write it, and every one of them is correct.

So do not read down this page hunting for your exact code. Instead, check your program against the **three things** below. If it does all three and it runs, **it is right** — even if it looks nothing like the version printed here.

The challenge, as set

Write a program that: (1) asks the user for their name; (2) asks them how many times they want to be greeted; (3) greets them that many times.

Check your program against these three things

1. **Two inputs, each stored in a variable.** The commonest miss is an `input()` whose answer goes nowhere — the program asks the question, throws the answer away, and then cannot use it.
2. **The number is cast to an integer.** ✓ **This is the one that is not style.** See the box on the next page.
3. **A loop that repeats the greeting exactly that many times.** Any loop. The greeting must be *inside* it — i.e. indented.

Everything else is style. Your variable names, your prompt wording, which loop you chose, how you joined the name to the greeting, whether the cast is on its own line. None of it is wrong. **There was no secret right answer.**

A canonical solution

```
name = input("What is your name? ")
times = int(input("How many times? "))
for i in range(times):
    print("Hello, " + name)
```

Four lines. **If yours is eight, that does not mean it is wrong** — longer code that works is still working code. Compare what it *does*, not how long it is.

Accepted variants — every one of these is correct

Every one of these was run and checked. They all produce exactly the same output.

1 A while loop with its own counter

A loop is a loop. for is tidier here, but while is not wrong — and if you reached for it, you worked out the mechanism instead of copying Program 5.

```
name = input("What is your name? ")
times = int(input("How many times? "))
count = 0
while count < times:
    print("Hello, " + name)
    count = count + 1
```

2 A comma instead of +

Two ways to join text to a variable. We use + in class, because that is what the exam uses — but a comma is not wrong, and it quietly puts a space in for you without your asking.

```
print("Hello, " + name)
print("Hello,", name)
```

3 The cast on its own line

Identical to the nested version, and arguably clearer: “get the text, then turn it into a number” is two ideas, written as two lines.

```
times = input("How many times? ")
times = int(times)
```

4 range(0, times) or range(1, times + 1)

All three loop the same number of times. If you wrote range(1, times + 1) you probably wanted the count to start at 1 — which is exactly the question Program 5 asked you about why it starts at 0.

```
for i in range(0, times):
for i in range(1, times + 1):
```

The one that looks like variant 4 and is not

```
for i in range(0, times + 1):
```

One greeting too many, every time — and almost nobody notices, because with times = 3 it prints four “Hello”s and nobody counts. **It only becomes obvious at zero**, where a request for no greetings produces one.

Written as	At times = 0	Verdict
range(times)	[]	correct
range(1, times + 1)	[]	correct
range(0, times + 1)	[0]	off by one

So test your own program with zero. If you ask for no greetings and get one, you have this bug. It is the fastest check there is, and it needs no reading.

Also completely fine: any variable names (n, num, howmany); any prompt wording; blank lines between sections; a print() before the questions to introduce the program; calling the loop variable something other than i. None of it changes what the program does.

The one thing that is not a variant — the cast

Without `int()`, `input()` hands back **text**, and `range()` cannot count to a piece of text. The program does not limp — it stops:

```
times = input("How many times? ")
for i in range(times):
    print("Hello, " + name)
```

TypeError: 'str' object cannot be interpreted as an integer

This is the same trap as Program 3, wearing a different hat. There, adding 1 to an `input()` failed. Here, counting to an `input()` fails. Same cause, and it is the single most useful thing on the page: **`input()` always gives you text.**

Note it is a *runtime* error, not a *syntax* error. The code is spelled perfectly. Python reads it, accepts it, starts running it, asks both questions — and only then falls over. Hold on to that: the table below turns on exactly this distinction.

Break it on purpose — the answers

If your wording is slightly different, you are still right. The browser IDE and desktop Python phrase these messages differently. **What matters is the *kind* of error, not the exact words.**

Do this to a working program	What the console said	Which kind
Delete a closing bracket	SyntaxError: unexpected EOF while parsing	Syntax
Spell print as prnt	NameError: name 'prnt' is not defined	Runtime
Remove the colon after if	SyntaxError: invalid syntax	Syntax
Add 1 to an <code>input()</code>	TypeError: can only concatenate str (not "int") to str	Runtime

Two syntax, two runtime. That is the “two of them are the same kind” the sheet promised — and it is true twice over.

The trap — and it catches teachers as well as students

A misspelled function name is not a syntax error.

`prnt("hello")` is *perfectly good Python grammar*. It is a name, followed by brackets, followed by a string — exactly the shape of every working line they wrote today. Python parses it without complaint, starts the program, reaches that line, goes looking for something called `prnt`, finds nothing, and stops. That is a **NameError**, and **NameError** is a **runtime** error.

Lots of revision websites get this wrong, and file a typo under “syntax error” because it feels like a spelling mistake. The test is not *does it look like a typo*. The test is **did the program start?** If it printed anything at all, or asked you a question, or got as far as line 3 — it started. So it is a runtime error.

And none of the four is a logic error. A logic error runs perfectly and gives the *wrong answer*, so it can never appear in a table of things that visibly break. If you wrote “logic” in the third column, that is an interesting mistake and worth understanding — come and ask.

One last thing — try it with zero

Run your program again and ask for **zero** greetings.

A correct program prints nothing at all. If yours prints one “Hello”, you have the off-by-one

above — and until now you had no way of knowing.

This is the habit worth taking away from the whole sheet. A program that works for 3 can still be wrong. **Test the edges**, not just the middle.